

Streaming Service

Made by

Jakob Heidrup Husen: jahu1000@stud.ek.dk

&

Malte Bjørklund: mabj0002@stud.ek.dk

Github:

https://github.com/GruppeNice/DLS_Exam

1. Introduction.....	3
1.1. Problem description.....	3
1.2. Functional and non-functional requirements.....	3
1.3. Explanation of choices for the technology stack (databases, programming languages,.....	4
2. System architecture.....	7
2.1. Introduction to the microservices architecture + architectural diagram of the whole.....	7
system.....	7
2.2. Microservices Description.....	7
2.3. Communication between microservices.....	9
2.4. Description of the patterns and techniques used in the project.....	9

1. Introduction

1.1. Problem description

This assignment consists of designing and implementing a streaming service using a microservices-based architecture organized within a single monorepo. The purpose is to model a simplified, real-world streaming platform where independent services collaborate to deliver a cohesive user experience while maintaining clear separation of responsibilities.

The system shall be decomposed into seven distinct microservices: User Management, Catalog (GraphQL-based), Streaming/Playback, Payment & Billing, Review & Rating, Recommendation (AI-based), and Engagement. Each microservice shall own its data, expose well-defined APIs, and operate as an independent deployable unit, even though all services are maintained within the same repository. Services must communicate through clearly defined interfaces and, where appropriate, event-driven mechanisms.

The platform shall support core streaming functionality, including user authentication and account lifecycle management, content discovery, streaming session handling, subscription management, user feedback, personalized recommendations, and automated engagement workflows triggered by system events. The assignment emphasizes proper service decomposition, data isolation, inter-service communication, and scalable system design.

1.2. Functional and non-functional requirements

Functional - 7 microservices:

1. User Management Service - The system shall be responsible for identity, authentication, and account lifecycle management.
2. Catalog Service (GraphQL) - The system shall manage all content metadata and search capabilities.
3. Streaming / Playback Service - The system shall handle streaming session lifecycle and playback tracking.
4. Payment & Billing Service - The system shall manage subscription plans and payment processing.
5. Review & Rating Service - The system shall manage user-generated feedback and content reputation.
6. Recommendation Service (AI) - The system shall generate personalized content recommendations based on behavioral data.
7. Engagement Service - The system shall handle user engagement workflows triggered by system events.

Non-functional:

1. The system shall support versioning to ensure backward compatibility between service updates
2. The system shall follow separation of concerns giving clear separation between presentation, business logic and data access layers
3. The system shall ensure low coupling by only communicating through well defined interfaces
4. The code shall adhere to coding standards and good practices including consistent naming conventions, removal of unused code, and comments for complex logic where appropriate.

1.3. Explanation of choices for the technology stack (databases, programming languages,

Frontend

- **React + TypeScript**
The UI framework. React handles the component-based interface, TypeScript adds static typing to catch errors at compile time rather than runtime.
- **Vite**
Build tool and development server. Significantly faster than the older Create React App, handles bundling for production.
- **Axios**
HTTP client for making API calls from the frontend to backend services — both REST and GraphQL endpoints.
- **React Query**
Manages server state in the frontend — handles caching, background refetching, and loading/error states for API responses so you don't have to manage this manually.

Backend

- **Java + Spring Boot**
The backend framework used across all Java microservices. Spring Boot removes most of the boilerplate configuration and provides a production-ready starting point for each service.
- **Spring Cloud Gateway**
The API gateway — a single entry point that routes requests from the frontend to the correct microservice. Also handles rate limiting and can enforce auth before requests reach the services.
- **Spring Security + JWT**
Authentication and authorization across all services. JWT (JSON Web Tokens) lets services verify a user's identity without hitting a database on every request.
- **Google OAuth 2.0**
Allows users to log in with their Google account instead of creating a password. Handled by the User Management Service.
- **Spring for GraphQL**
GraphQL support for the Catalog Service, which exposes a GraphQL API for flexible content queries from the frontend.
- **Spring Data JPA + Hibernate**
The database access layer for all Java services. JPA provides a standard interface, Hibernate is the underlying ORM that translates Java objects to SQL queries.
- **springdoc-openapi (Swagger)**
Automatically generates Swagger UI documentation for all REST APIs directly from the code. Required by the assignment — one spec per service, accessible via /swagger-ui.html.

AI / ML — Recommendation Service

- **Python + Scikit-learn**
The ML framework used to build the recommendation model. Scikit-learn provides collaborative filtering algorithms that match users to content based on behaviour patterns — the right tool for this problem.
- **FastAPI**
Exposes the Python ML model as a lightweight REST API so the backend Java services can call it. Chosen over Flask because it is faster and generates API docs automatically.
- **Pandas + NumPy**
Data processing libraries used to prepare and transform playback and rating event data before feeding it into the recommendation model.
- **Ollama (containerized)**
A local LLM that can be run as a container with no external API or cost. The assignment explicitly permits this as an alternative or complement to Scikit-learn — useful if the group wants to add generative features such as content descriptions or a search assistant.

Messaging

- **RabbitMQ**

The message broker that enables asynchronous communication between services. Required by the assignment as the primary method of service-to-service communication — services publish events to queues rather than calling each other directly.

Database

- **MySQL**

Relational database used by all services. Each microservice owns its own separate database instance — services are not allowed to query each other's databases directly, which is a core architectural requirement.

- **Flyway**

Manages database schema changes as versioned migration files stored in the repository. Required by the assignment as part of the database versioning strategy — ensures all team members and environments stay in sync.

Containerisation & Orchestration

- **Docker + Docker Compose**

Each service is packaged as a Docker container with all its dependencies. Docker Compose defines and runs the full local development environment — all services, databases, and RabbitMQ — with a single command. Required by the assignment.

- **Minikube or k3s**

Runs a local Kubernetes cluster on your machine to simulate a production environment. Required by the assignment — full cloud deployment is not needed, but the system must run on local Kubernetes.

- **KEDA (Kubernetes Event-Driven Autoscaling)**

Deployed in the Kubernetes environment to run the Engagement Service as a ScaledJob — a background job that scales up when there are messages in the queue and scales back to zero when idle. Required by the assignment: at least one serverless-style isolated task must be implemented.

CI/CD

- **GitHub Actions**

Automates the build, test and deployment pipeline on every commit. Runs all tests, builds Docker images, and pushes them to the container registry. Required by the assignment.

- **Docker Hub or GCP Artifact Registry**

Stores the built Docker images so they can be pulled by the Kubernetes cluster during deployment.

- **SonarCloud or SpotBugs**

Static analysis for Java services — scans for code quality issues and security vulnerabilities automatically as part of the CI/CD pipeline. Required by the assignment.

- **Bandit**

Static security scanner for the Python ML service. Runs in CI/CD alongside the Java scanners. Required by the assignment.

- **ESLint**

Static analysis for the React/TypeScript frontend — enforces code style and catches common errors. Runs in CI/CD.

Observability

- **Prometheus**

Scrapes metrics (request count, error rate, response times, memory usage) from all services at regular intervals. Spring Boot's Actuator module automatically exposes a /metrics endpoint that Prometheus reads — minimal setup needed in the application code.

- **Grafana**

Visualises the metrics collected by Prometheus as dashboards. Also serves as the UI for Loki logs — so one tool covers both metrics and log viewing.

- **Grafana Loki**
Collects and stores logs from all services in one place. Instead of SSH-ing into individual containers to read logs, all service output is searchable from Grafana. Services are configured to ship their logs to Loki — no changes needed in application code.
- **Zipkin**
Distributed tracing — tracks a single user request as it travels through multiple services and shows a timeline of exactly where time was spent. Added to Spring Boot via the Micrometer Tracing dependency, which automatically instruments requests with trace IDs.

Testing

- **JUnit 5 + Mockito**
Unit testing framework for Java services. JUnit runs the tests, Mockito mocks dependencies so each unit is tested in isolation.
- **Testcontainers**
Spins up real Docker containers (MySQL, RabbitMQ) during integration tests, so tests run against actual infrastructure rather than mocks. Used for database integration tests, messaging tests, and system-level cooperation tests.
- **Spring Security Test**
Extension of the Spring testing tools specifically for testing authentication and authorization — verifies that protected endpoints reject unauthorized requests correctly.
- **Pytest**
Testing framework for the Python ML service — covers unit and integration tests for the recommendation logic.

Versioning & Documentation

- **Git + GitHub (monorepo)**
All microservices live in a single repository. Individual contributions are tracked via git history — required by the assignment so each team member's work is clearly visible.
- **URI versioning — /api/v1/**
All REST endpoints are prefixed with a version number. This is the versioning strategy for REST APIs — required by the assignment. GraphQL does not require versioning by design.
- **Flyway migration files (see Database section)**
Schema changes are stored as numbered SQL migration files alongside the source code — this doubles as the database versioning strategy.
- **Swagger / OpenAPI via springdoc (see Backend section)**
Auto-generated REST API documentation. Mentioned here as it fulfils the REST documentation requirement.
- **AsyncAPI specification**
Documents all RabbitMQ communication — queues, message schemas, which services produce and consume each event. Required by the assignment. AsyncAPI Studio is a free web tool for writing and visualising these specs.
- **GitHub README + Wiki**
Each microservice has its own README covering purpose, setup and API summary. The monorepo root README covers the overall system. Required by the assignment.

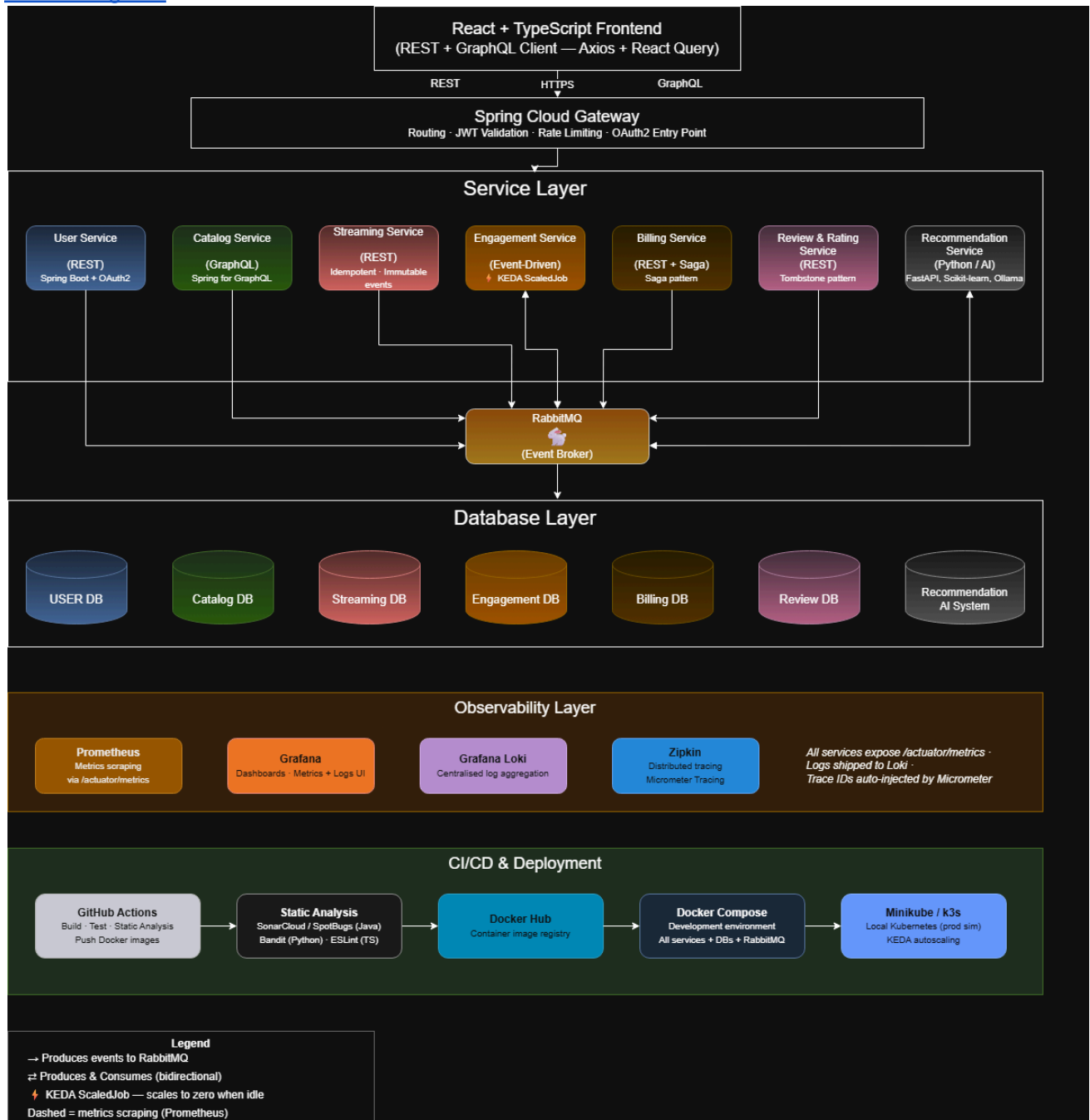
Cloud Provider

- **GCP or AWS**
Either is a strong choice. GCP offers Cloud SQL (managed MySQL) and GKE (managed Kubernetes). AWS offers RDS (managed MySQL) and EKS (managed Kubernetes). The report must describe which services would be used, pricing, scalability and availability considerations.

2. System architecture

2.1. Introduction to the microservices architecture + architectural diagram of the whole system

[Link to Diagram:](#)



2.2. Microservices Description

	Responsibilities	Data ownership	API	Events Produced
User Management Service:	- User registration and login - JWT token issuance - OAuth integration (e.g., Google) - Password reset workflow - Account status management (active, suspended, banned) - Basic profile data (email, display name)	Users Credentials Roles Account state	REST	- Event - UserRegistered - UserSuspended - UserDeleted
Catalog Service (GraphQL):	- Movie and TV show metadata - Genres, tags, categories - Cast and crew information, release dates - Region-based availability - Content search and filtering	Content metadata Availability rules Search indexes	GraphQL	- Produced - Consumed - ContentCreated - ContentRated - ContentUpdated - ContentReviewed - ContentRemoved
Streaming / Playback Service:	- Start / stop / resume playback sessions - Track watch progress - Validate active subscription before streaming - DRM validation hook (simulated)	—	REST	- Event - PlaybackStarted - PlaybackStopped - PlaybackProgressUpdated
Payment & Billing Service:	- Subscription plan definitions and activation / cancellation - Payment processing and invoice generation - Refund handling - Payment gateway integration (simulated)	Subscription records Payment transactions Invoices	REST	- Event - SubscriptionActivated - SubscriptionCancelled - PaymentSucceeded - PaymentFailed
Review & Rating Service:	- Create / update / delete ratings (1–5 stars) - Written reviews and basic moderation status - Aggregate rating calculation	Ratings Reviews Moderation flags	REST	- Event - ContentRated - ContentReviewed - ReviewModerated
Recommendation Service (AI):	- Consume playback and rating events - Build user preference models - Generate personalised "Recommended for You" lists - Calculate trending / popular content rankings - Periodic model retraining	Recommendation models User preference vectors Precomputed recommendation lists	REST	- Event - PlaybackStarted - PlaybackProgressUpdated - ContentRated - SubscriptionActivated
Engagement Service:	- Generate email, push, and in-app notifications - Personalised "Continue Watching" reminders - Re-engagement campaigns and delivery tracking	—	Async-driven (primary) Minimal REST for manual triggers	- Event - SubscriptionActivated - PlaybackStopped - ContentCreated

2.3. Communication between microservices

Asynchronous messaging via RabbitMQ is used as the main mechanism for inter-service communication for event-driven workflows. Services publish domain events to RabbitMQ, and other services subscribe to the events they need. This reduces direct service-to-service dependencies and improves resilience by decoupling producers from consumers. Current consumers include the Recommendation Service and the Engagement Service, with a future Analytics Service planned.

In addition to asynchronous messaging, some interactions require immediate responses and are therefore implemented using synchronous REST communication. For example, when a user starts a playback session, the Streaming Service verifies that the user has an active subscription by calling the Billing Service. This ensures that access control decisions can be made immediately while still keeping the overall architecture event-driven.

Asynchronous Event Flow

Producer	Events
Streaming Service	PlaybackStarted, PlaybackStopped, PlaybackProgressUpdated
Review Service	ContentRated, ContentReviewed
Billing Service	SubscriptionActivated, SubscriptionCancelled, PaymentSucceeded, PaymentFailed
Catalog Service	ContentCreated, ContentUpdated, ContentRemoved

Engagement-service:

The engagement-service is responsible for asynchronous engagement-related processes, with a primary focus on sending notifications.

Goal: Asynchronous notification service (email/SMS)

API: REST endpoint to create a notification request

Template Engine: Renders message content from templates + variables

Queueing: Publishes notification ID to **RabbitMQ** (*notification-queue*)

Consumer/Worker: *@RabbitListener* processes queued notifications

Sending: Pluggable *NotificationSender* by type (EMAIL, ...)

Retry: Automatic retries with backoff (*@Retryable*), final *FAILED* via *@Recover*

Status Flow: *QUEUED* → *PROCESSING* → *SENT* / *RETRY* → *FAILED*

Email Testing: MailHog SMTP sink + web UI to view sent emails

Observability: Logs + DB status tracking (trace by notification ID)

Deployment: Docker Compose for RabbitMQ + MailHog (optional app containerization)

2.4. Description of the patterns and techniques used in the project.

Several distributed system patterns were selected to improve scalability and reliability. The Saga pattern is used in the Billing Service during subscription activation, where multiple operations such as payment processing, subscription creation and invoice generation must succeed as a single business transaction. Idempotency is used in both the Billing and Streaming services to prevent duplicate processing if a request is retried. Immutable events are published through RabbitMQ and act as a historical record of important business actions within the system.

Distributed Design Patterns

Pattern	Applied In
CQRS	Command-based Streaming · Query-based Catalog & Recommendations
Immutable Events	Playback events
Tombstone Pattern	Deleted reviews and content
Idempotency	Streaming and billing operations
Saga Pattern	Subscription activation workflow
Commutative Aggregation	Recommendations and trending calculations
Serverless / KEDA ScaledJob	Engagement Service background jobs
REST + GraphQL	Interoperability across services
AI Integration	Recommendation Service as real business workflow

3. Environments description

3.1. Development environment with docker-compose including the architecture diagram

The implemented services are designed to run as isolated microservices within local development environments using Docker Compose. At the time of writing, the User Service, Billing Service, and Streaming Service each provide their own Docker Compose configuration, including an application container, a PostgreSQL database, and a RabbitMQ broker. This setup allows developers to work on and test services independently while still following the same architectural principles across the system.

The environments are configured with health checks and startup dependencies, ensuring that applications only start once their required infrastructure components are available. This helps reduce startup issues during development and provides a more stable local environment.

From an architectural perspective, all services follow a similar request-processing flow. Incoming client requests are handled through REST APIs, business logic is executed within the service, and data is stored in a dedicated database owned by that service. The User Service handles registration, authentication, and account management. The Billing Service manages subscriptions, payment processing, invoice generation, and refund handling. The Streaming Service is responsible for playback sessions, watch progress tracking, and validating active subscriptions before playback begins.

In addition to database operations, services publish domain events through RabbitMQ, allowing other services to react asynchronously when business actions occur. This supports the event-driven architecture described earlier and reduces direct dependencies between services. Operational visibility is provided through Spring Actuator endpoints, which expose health and metrics information compatible with monitoring tools such as Prometheus.

3.2. Local Kubernetes deployment simulating a production-like environment including the architecture diagram

Although a full Kubernetes deployment has not yet been implemented, the User Service, Billing Service, and Streaming Service are all designed to be deployment-ready within a containerized environment. Each service is packaged using Docker and follows a stateless design, making them suitable candidates for orchestration through Kubernetes.

Configuration is externalized through environment variables and configuration files, allowing deployments to be adjusted without modifying application code. Dependencies such as databases and RabbitMQ are already separated from the application layer, which aligns with common microservice deployment practices.

A future Kubernetes deployment would likely consist of individual Deployment resources for each service, Service resources for internal communication, and ConfigMaps and Secrets for configuration management. Sensitive information such as database credentials and JWT secrets would be stored using Kubernetes Secrets rather than hardcoded configuration values.

The same deployment model can be applied consistently across the implemented services. Communication between services would continue to occur through well-defined APIs and RabbitMQ events, while database ownership remains isolated within each service boundary.

3.2.1. Description of used technologies

The User Service, Billing Service, and Streaming Service are all built using the same Java-based technology stack. Using the same technologies across services makes development easier and helps keep the project consistent.

All three services use Java 21 and Spring Boot 3.5. Authentication is handled using Spring Security and JWT tokens. Tokens are issued by the User Service and can be validated by the other services without requiring direct access to user data.

For data storage, each service has its own PostgreSQL database and uses Spring Data JPA together with Hibernate to access data. Flyway is used for database migrations, making it easier to keep database schemas synchronized across development environments.

RabbitMQ is used for asynchronous communication between services. When important actions occur, such as a subscription being activated or a playback session being started, events can be published so that other services can react without needing direct communication.

Spring Actuator is used for monitoring endpoints, while Springdoc OpenAPI is used for API documentation. AsyncAPI specifications are also included to document event-based communication. All services are containerized using Docker and run locally through Docker Compose.

Although the services use many of the same technologies, they each have their own responsibility. The User Service handles authentication and user accounts, the Billing Service handles subscriptions and payments, and the Streaming Service manages playback sessions and watch progress.

3.2.2. CI/CD pipeline description and diagram

While a dedicated CI/CD pipeline configuration is not yet implemented specifically for the User Service, the project structure supports a conventional pipeline model. The expected pipeline stages include compilation and build using Maven, followed by execution of automated tests.

Subsequent stages would involve static code analysis and security scanning, ensuring code quality and dependency safety. Once validated, a Docker image would be built and published to a container registry. Finally, deployment steps would update the target environment, potentially using rolling updates to minimize downtime.

This pipeline model aligns with standard practices and can be integrated into the broader project-level CI/CD strategy.

3.2.3. Monitoring and logging of the deployed system

The implemented services include built-in monitoring support through Spring Actuator. Endpoints such as health, info, metrics, and prometheus are exposed through application configuration, providing visibility into service health and runtime behaviour.

This design makes it possible to integrate all services with monitoring platforms such as Prometheus and Grafana. Metrics including request counts, response times, error rates, and application health can be collected consistently across the system, providing a unified monitoring strategy for the distributed architecture.

Although centralized logging has not yet been fully configured, the services are designed to support integration with logging solutions such as Grafana Loki. This would allow logs from multiple services to be collected and searched from a single location, simplifying troubleshooting and operational monitoring.

3.2.4. Autoscaling configuration

Autoscaling has not yet been explicitly configured for the User Service. However, the application's stateless design makes it inherently suitable for horizontal scaling in environments such as Kubernetes.

Scaling considerations differ across system components. The application layer can scale horizontally by increasing the number of service instances. In contrast, the database layer requires more careful capacity planning, potentially involving replication or managed database solutions. Similarly, message throughput can be adjusted by tuning RabbitMQ configuration and optimizing consumer behavior in downstream services.

3.3. Cloud deployment plan – deployment diagram, selected cloud providers and services, pricing strategies, scalability and availability considerations (both app and database level), security and data management strategies, deployment and release approaches, and justified architectural trade-offs. Actual cloud deployment is not required.

Although no cloud deployment has been executed, the User Service is designed with cloud readiness in mind. Its containerized architecture, externalized configuration, and clear separation of concerns allow for straightforward migration to cloud platforms.

A typical deployment strategy would involve running the service within a managed Kubernetes environment or container platform. PostgreSQL would be replaced by a managed database service to ensure durability, scalability, and backup capabilities. Similarly, a managed message broker or cloud-native messaging solution could be used in place of RabbitMQ.

Secrets such as JWT keys and database credentials would be handled through a dedicated secret management system. Deployment updates would follow rolling strategies, leveraging health checks to ensure service availability during updates.

4. Testing

4.1. Testing strategy and scope

4.2. Unit testing

4.3. Integration testing

4.4. System-level cooperation testing

4.5. Security testing focusing on verifying correct authentication and authorization

7

4.6. Static analysis and static testing to identify code quality or security issues.

4.7. Test execution in CI/CD

4.8. Limitations, risks, and trade-offs

5. Project management and team collaboration

5.1. Introduction to the project management and team collaboration

5.2. Description of the methods used during the project.

5.3. Versioning strategies for the source code, databases, and APIs

5.4. Documentation strategy

6. Discussion

6.1. Advantages and challenges of the distributed systems (application and database perspectives)

6.2. Pros and cons of used patterns like CQRS etc.

6.3. Scalability / Autoscaling (application, messaging, and database perspectives)

6.4. Possible improvements

7. Reflection: What was difficult about: designing the distributed system, project management, what design mistakes were made and fixed, what would be done differently next time, what you learned about databases in practice.

8. Conclusion: short high-level summary of the project, results, main insights.

9. References: All major design choices, technical claims, and comparisons must be supported by references. Sources must be cited in the text and listed in the References section.

References should primarily be official documentation, textbooks, or academic or technical publications. Web tutorials and blogs may be used sparingly but should not be the primary sources. Use standard formats like APA, IEEE, or Harvard.

10. Appendix